GECKO: a fast, explainable system for analyzing and optimizing keyboard layouts

How the scorer, the optimizers, and the analysis tools work, and what you can do with them.

Joshua Markle

University of Virginia · October 2026

Working draft · describes the main branch of 2026-10-21 · numbers marked TODO are not yet measured

Abstract

Keyboard layout design is a combinatorial search ($30! \approx 2.65 \times 10^{32}$ placements of 30 letters) wrapped around a judgement problem: what makes a layout comfortable? Existing analyzers fix one answer to the second question, hide the per-metric reasons behind one score, and cannot say how close a layout is to the best possible. GECKO is an open-source Rust system (core library, command-line tool, server, WebAssembly build) with a TypeScript web UI that separates the two problems. A *scorer* turns a board geometry, a text corpus, and a weighted profile of 48 metrics into pre-folded cost tables, so evaluating a key swap touches only the n-grams that contain the swapped symbols: 6.4×10^5 swap evaluations per second per core under the default profile, and 5.0×10^6 per second for a search on 20 threads. Eight discrete optimizers share one trait and one threaded runner. Every run records its seed, input hashes, and parameters, and can be re-verified. Around the search sit tools to explain a score (per metric, finger, n-gram, and word), to bound the optimum, to map the landscape of good layouts, to trade one goal against another, and to cost a layout on a different board, with layers and adaptive keys. We describe the design and the invariants that keep it correct, measure its speed, and walk through six concrete uses.

1. Introduction

Alternative layouts (Dvorak, Colemak, Workman, Graphite, Sturdy, Semimak, Canary, and thousands more in community collections) are designed with a family of analyzers and optimizers: carpalx, KLO, Kühlmak, genkey, Oxeylyzer, and cmini's analyzer, among others. They agree on the core ideas (penalize same-finger bigrams, lateral stretches, scissors, and awkward trigrams; balance load) and disagree on weights, definitions, and the search used. Three problems recur.

The score is opaque. One number hides which trade-offs it made, and two tools rank the same layouts differently with no way to tell why. **The optimizer is welded to the objective.** Changing a metric means editing the search. **Nobody says how good "good" is.** A heuristic returns a layout, with no certificate and no map of what else was nearly as good.

GECKO is built for four kinds of user: people choosing or designing a layout for their own hands, language, and board; researchers who need a layout judge they can inspect and reproduce; keyboard builders who want to compare boards; and scripts and agents, for which every command has a stable JSON output. Its contributions are:

- a scorer built for swaps: all metrics fold into one table per n-gram class, so adding a metric to an existing table costs nothing at search time (Section 4);
- one trait for the discrete optimizers, compared on equal terms by `gecko bench` (Section 5);
- explanations, certificates, and maps: every score decomposes per metric, finger, n-gram, and word; lower bounds give optimality gaps; atlases and quality-diversity grids show what else is nearly as good (Section 6);
- geometry in the loop: a parametric board model and a hand model whose costs fold into the same tables (Section 7);
- six worked uses, from auditing a reference layout to designing for a split board with thumb keys (Section 8);
- reproducibility by construction: a seeded RNG identical on native and WebAssembly, content hashes, and `gecko verify` (Section 10).

2. System overview

GECKO is a Cargo workspace of three crates and a web app. `gecko-core` (about 19k lines of Rust) holds everything that matters: geometry, corpus, metrics, scorer, optimizers, bounds, and reports. It does no I/O

beyond reading and writing data files. `gecko-cli` is the `gecko` binary, including `gecko serve`, an HTTP server with a JSON API and server-sent progress events. `gecko-wasm` wraps the same core for the browser, so the web UI scores a layout with the code the CLI uses. The web UI (TypeScript, React, Three.js) has an input side (Board, Hand, Language, Profile, Search, Summary) and a results side (Home, Explore, Analysis, Saved, Export), with a live layout editor and a 3D view of the board.

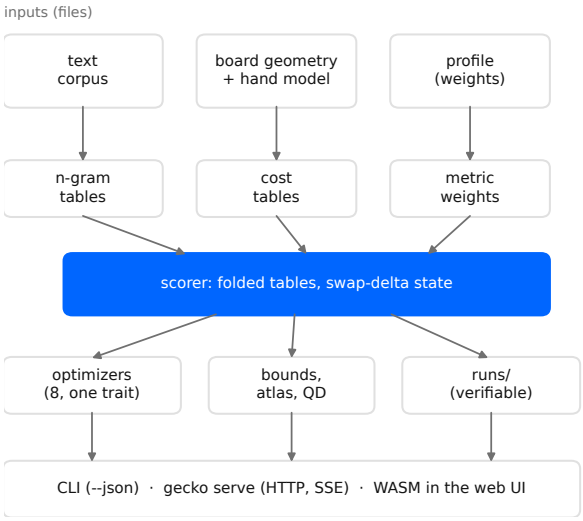


Figure 1: Data flow. A corpus becomes n-gram tables; a geometry (with the hand model) becomes cost tables; the profile weights them. The scorer folds these once, and every algorithm and front end reads it.

Inputs are four kinds of file. A *geometry* (TOML; explicit or parametric; importable from keyboard-layout-editor.com) lists key slots with position, rotation, finger, and effort. A *corpus* is text compiled to a binary n-gram table (`.gkc`); bundled corpora include English, French, code, and Reddit text, and corpora can be mixed with weights. A *profile* (TOML) gives metric weights in spread units (Section 4) plus finger-load targets and hand parameters; seven are bundled (default, code, sfb, engram, familiar, hand, learned). A *layout* is a slot-ordered list of symbols, optionally with layers and adaptive keys. Any command accepts a layout as a name, a TOML path, a literal (`raw:qwerty…`), a saved run (`run:<id>#rank`), or an imported community layout (`cmini:<name>`).

Outputs are run directories. Every search writes `runs/<time>-<algo>-<hash>/` with its configuration, input hashes, progress log, and ranked results (Section 10). The web UI lists, compares, and re-opens them, so a result found on the command line can be inspected in 3D and vice versa.

About 3k lines of Rust and 7.6k of TypeScript sit outside the core; 197 Rust tests (including property tests) and 69 web tests run in CI.

3. Problem formulation

A *geometry* is a list of N slots (position, rotation, size, finger, row, column, home flag, effort). A *layout* assigns a symbol to each slot; empty slots, thumb tokens, and layer keys are allowed. A *corpus* supplies frequencies f_1 (symbols), f_2 (bigrams), f_2^k (skipgrams, gaps 1–3), and f_3 (trigrams). A *profile* holds metric weights and parameters. With $p(x)$ the slot of symbol x, the objective is

$$\begin{aligned}\text{score}(p) = {} & \textstyle\sum_x f_1[x]\, C_1[p(x)] \\ & + \textstyle\sum_{x,y} f_2[x,y]\, C_2[p(x),p(y)] + \text{skipgrams} \\ & + \textstyle\sum_{x,y,z} f_3[x,y,z]\, C_3[p(x),p(y),p(z)] \\ & + \text{non-linear terms.}\end{aligned}$$

The first three lines are a quadratic assignment problem (QAP) with a sparse cubic part. The last line holds functions of a few per-finger accumulators (finger balance, peak load, hand balance). Swapping symbols a and b changes only terms that contain a or b.

4. The scorer

4.1 Metrics as cost tables

A metric is a *table builder*: given a geometry it returns a per-slot, per-pair, or per-triple cost, plus a rule for reporting a raw value (a percentage or a per-1000 rate). GECKO implements 48 metrics in families: same-finger (SFB, SFS, far SFS, distance); stretch and scissors (a geometric stretch adapted from Oxeylyzer 2, a finger-pair movement table); rolls and alternation (a dozen trigram classes, including thumbs); load and balance; comfort judges (the Engram formula and measured tables); hand-model costs (reach, wrist posture, press angle); and symbol and layer metrics (layer switches, code bigrams, paired brackets, home-row-mod misfires, shortcut placement, familiarity to a reference layout).

4.2 Folding

For a profile with weights w_m, the linear metrics are summed once at build time, for example $C_2[s][t] = \sum_m w_m\, c_{2,m}[s][t]$, and likewise for C_1, the skipgram table, and C_3. Search cost therefore does not depend on how many metrics feed a table. Weights are expressed in *spread units*: each is divided by the metric's standard deviation over the reference layouts, so a weight reads as "how many reference spreads of this metric I trade for one of another." Raw, unweighted values are recomputed separately for reports.

4.3 Swap-delta evaluation

For a swap of a and b, the change is computed from the $O(S)$ bigram partners of each symbol and the sparse list of non-zero trigrams that contain them. Trigrams are stored per symbol; a `State` caches the folded cost of every non-zero trigram, so a delta reads the old cost

and `apply` refreshes only touched entries. The hot loop performs no allocation, no hash lookups, and no dynamic dispatch. The cache stores exact unweighted `f32` costs so that caching does not change the arithmetic. An index-based CSR variant was tried and rejected (about 305k versus 370k deltas per second). Non-linear terms update in $O(1)$ from per-finger accumulators.

Table 1: Single-thread speed (ANSI-30, English, default profile, i9-13900H; `gecko perf`, 2026-10-21).

Operation	Rate (/s)	Time
Full evaluation	158,101	6.33 µs
Swap delta	638,805	1.57 µs
Scorer build	334	3.00 ms
Move one key, rebuild	1,551	0.64 ms

A delta is about $4\times$ faster than a full evaluation under the default profile. How much a profile costs depends on which *channels* it switches on, not on how many metrics it has (Figure 2). We enabled metrics in a fixed order and measured swap deltas after each addition. Throughput falls in three steps: from 4.8×10^6 per second with three bigram and gap-1 skipgram metrics to 2.4×10^6 when the first far-skipgram metric (S02, gaps 2–3) switches on its channel, to 2.1×10^6 when the load and balance terms add their accumulators, and to 0.64×10^6 when the first trigram metric adds the sparse trigram list. Between steps the curve is flat: metrics 4 to 13 and metrics 25 to 35 cost nothing extra. The rule of thumb for profile authors is that trigram metrics set the price of a search and the number of bigram metrics does not.

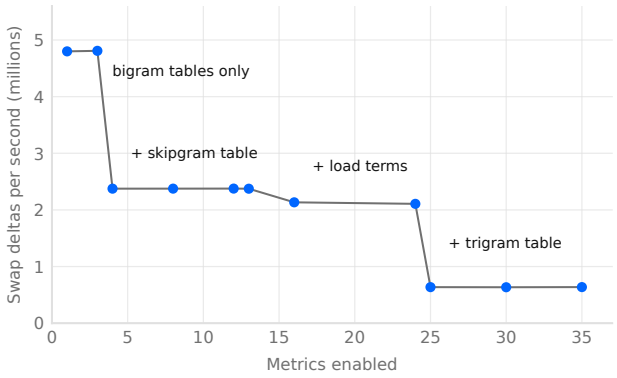


Figure 2: Swap-delta throughput against the number of enabled metrics (one thread, ANSI-30, English; median of 5 runs; script `metrics_scaling.py`). Labels name the channel each step switches on.

The runner scales linearly to 4 threads (Figure 3): $4.07\times$ the one-thread rate. It then bends as efficiency cores and hyperthreads join: 12 threads reach $8.3\times$ and 20 threads $8.7\times$ (5.0×10^6 evaluations per second; the CPU has 6 performance and 8 efficiency cores). The 8-thread points are noisy because background processes shared the machine; a quieter rerun would tighten them.

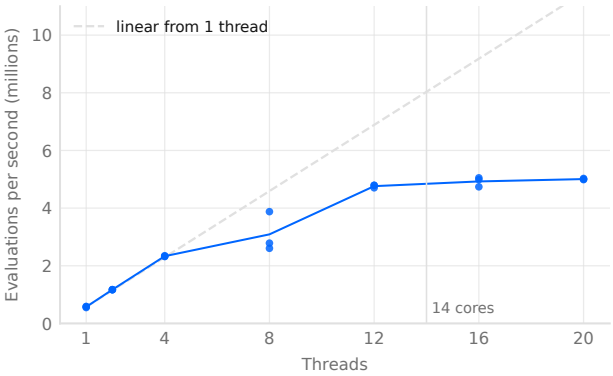


Figure 3: Evaluations per second of simulated annealing against threads (3 seeds per point, 4 s each; ANSI-30, English, default profile; i9-13900H with 6 performance and 8 efficiency cores; script `thread_scaling.sh`).

4.4 Invariants

Correctness rests on tests that run in CI.

- **delta = full**: for random layouts and swaps, the delta equals the re-evaluation within 10^{-5} relative, including 36-key boards with tokens and empty slots (property test).
- **folded = unfolded**: the folded score equals the sum of individually computed metrics.
- **hand-computed metrics**: each metric on 3–6 key boards with answers worked by hand.
- **ranking and published numbers**: modern layouts rank above Colemak, Dvorak, and QWERTY; SFB values are within tolerance of published ones.
- **brute force**: every optimizer finds the optimum on instances of at most 8 keys.
- **rebuild = build**: re-costing a moved board equals building it from scratch, bit for bit.
- **native = WASM**: every reference layout under every profile scores the same in the browser and the CLI (relative 10^{-6}).
- **determinism**: the same seed gives the same result.

5. Optimizers

All discrete optimizers implement one trait:

```
trait Optimizer {
  fn init(&mut self, scorer, start, rng);
  fn step(&mut self, scorer, rng)
        -> StepInfo;      // a batch of work
  fn best(&self) -> &[Scored<Layout>];
}                           // de-duplicated top-K
```

The runner owns time budgets, threads, progress events, and persistence; algorithms contain only search logic. Workers run on scoped standard-library threads. The scorer is shared immutably, and each worker owns its state and an RNG seeded from the run seed and the worker index. A run can be paused, resumed, or cancelled without losing results. Any key

can be pinned (for example the thumb keys), and every move respects pins.

Table 2: The optimizers.

Algorithm	Role
Hill climbing	baseline; inner loop of the others
Simulated annealing	main workhorse; T_0 set from sampled swap deltas
Breakout local search	descent plus adaptive kicks; samples the local-optima network
Tabu (RoTS)	robust tabu search for QAP
Genetic, memetic	crossover plus local search on islands
MAP-Elites	quality-diversity grid over behaviours
NSGA-II, random-weight SA	Pareto fronts over chosen metrics
CMA-ES (IPOP)	continuous geometry parameters (outside the trait)

Beyond swaps, `--moves` adds same-finger column exchange, 3-cycles, and depth-2 swaps. All are swap sequences, so they reuse the delta machinery and respect pinned keys. Larger moves give better optima per descent but cost 3–60× more evaluations, so swap-only stays the default.

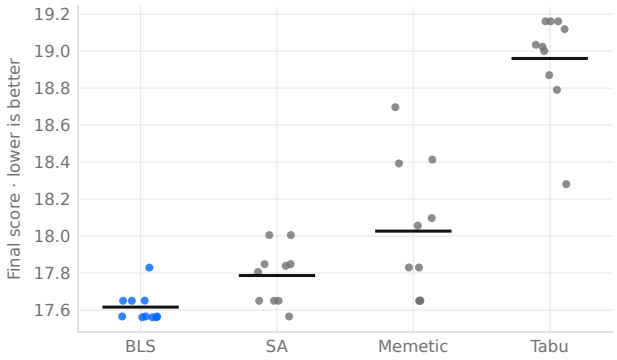


Figure 4: Final score per seed for four optimizers: 10 seeds, 2 s, 4 threads, ANSI-30, English, default profile (`bench/memetic-2026-10-21`). Black bars are means; background builds were running.

On this short budget BLS has the lowest mean (17.616 ± 0.081), followed by SA (17.787 ± 0.145), memetic (18.027 ± 0.352), and tabu (18.960 ± 0.256). This is one budget and one instance; TODO(run): a budget grid with 30 seeds, performance profiles, and time-to-target curves before drawing general conclusions.

6. Explaining, bounding, and mapping

6.1 Explaining a score

For any layout, `gecko score` and `analyze` and the web UI report each metric's weighted contribution and raw value, finger and hand loads, a per-key heat map, the worst bigrams, skipgrams, and trigrams, and (with a word corpus) the worst real words per metric. Reference layouts are scored in the same pipeline and checked against published numbers (QWERTY SFB 6.63%). The web UI is the same core compiled to WebAssembly, so a number shown while dragging a key equals the CLI's.

6.2 Lower bounds and optimality gaps

The pair part of the score is a QAP. Each trigram is relaxed into pair terms by taking, for each pair, the minimum over the free third key, so the result remains a lower bound. The bound is Gilmore–Lawler (a linear assignment for each placement, via the Hungarian algorithm) strengthened by 20 iterations of dual ascent. Every optimization run reports the gap, $(\text{value} - \text{lower})/|\text{value}|$.

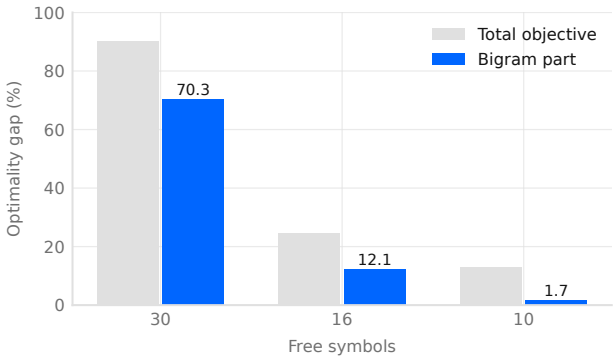


Figure 5: Optimality gap of the SA optimum against the number of free symbols (pinned symbols are constants). ANSI-30, English, default profile, run `20261021-171218-sa-a3306812`.

With all 30 symbols free, the bound certifies little (90.3% gap). With 10 free symbols the bigram part is certified within 1.7%; the remaining total gap there (12.9%) comes mostly from the trigram relaxation. Dual ascent closed about 1.3 of 14.8 points on a pair-only profile, so the weakness is in the bound, not the search. Tighter bounds would need stronger relaxations; we report this limitation rather than hide it.

6.3 Local-optima atlases

`gecko explore` samples the random-score distribution and random-walk autocorrelation, then runs breakout local search with an archive that stores every local optimum with its full raw metric vector, and every kick as an edge of a local-optima network. *Exclusion restarts* begin new descents at least 8 slots from the known best, to find layouts different from the winners. Because the archive stores raw vectors, it can be *re-ranked under new weights in milliseconds* without a new search, which makes weight-sensitivity studies cheap. `gecko atlas` reports families, consensus slots, funnels, and fitness–distance correlation.

6.4 Quality diversity and Pareto fronts

`gecko qd` runs MAP-Elites over behaviour axes (roll ratio, index load, pinky load, hand balance, or any

metric id). Each cell of the grid holds the best layout with that behaviour, so a user can ask for "more rolls" or "less index load" and see the price (Section 8). NSGA-II and random-weight SA provide Pareto fronts over chosen metrics.

7. Geometry in the loop

Boards are TOML files (explicit, or parametric: zones, columns, rows, stagger, splay, split, thumb arcs), importable from keyboard-layout-editor.com, with eight bundled boards (ANSI-30, ortho-30, Corne, Sweep, Lily58, Kyria, ErgoDox, Dactyl-Manuform 5×6). Geometry-dependent costs come from derived effort and stretch, and from a *hand model*: a skeleton placed on the home keys, planar finger inverse kinematics with joint limits, a 4-DoF thumb, and arm geometry. It produces per-key costs for reach, press-angle alignment, and wrist posture. These are per-slot tables, so they fold into the scorer with no search-time cost. In the web UI the hand has size presets and every segment length can be edited, so a score can be tied to one person's hand.

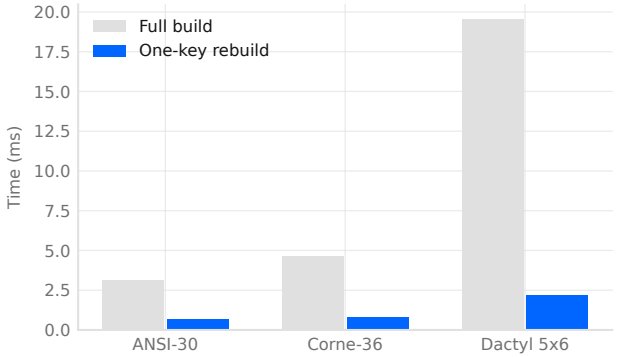


Figure 6: Time to re-cost the scorer after moving one key, against a full build (`gecko perf`, English, default profile).

`Scorer::set_geometry` recomputes only the tables whose inputs moved and is bit-identical to a full rebuild. Moving one key costs 0.64 ms against 3.1 ms on ANSI-30 ($4.8\times$), 0.78 ms against 4.6 ms on Corne, and 2.2 ms against 19.5 ms on the 3D Dactyl-Manuform ($9\times$). This is what makes bilevel search (an outer CMA-ES over board parameters, an inner layout search) feasible. A 2D collision test (oriented-box separating axis) prevents overlapping keys. Three-dimensional bilevel search and collision are in progress.

8. Using GECKO

Each use below is a question, the commands that answer it, and what they returned on the bundled data (ANSI-30 or Corne, English, default profile unless stated). The runs, and the scripts behind Figures 2 and 3, are in `bench/software-paper-2026-10-21/`.

Table 3: Uses at a glance.

Question	Commands	You get
Why does a layout rank where it does?	`compare`, `score`, `analyze`	per-metric and per-word costs
Best layout for my board and language?	`optimize`	ranked layouts, bound, holdout check
How good is "good"?	`landscape`, `explore`, `atlas`	random baseline, local optima, families
What does a goal cost?	`qd`, `optimize --algo nsga2`	best layout per behaviour cell
Can a layer or adaptive key help?	`score`, `magic`	score with rules applied
Is this result reproducible?	`verify`, `layouts`	pass or fail; exchange files

8.1 Audit a layout and see why it ranks

`gecko compare` ranks the ten bundled reference layouts under one profile and one corpus (Figure 7). Under the default profile the order is Graphite, Sturdy, Semimak, Canary, Dvorak, Colemak-DH, Halmak, Colemak, Workman, QWERTY. This ranking is a property of the profile, not a claim about comfort, and the per-metric table says what drives each gap: Colemak and Colemak-DH have identical SFB (1.54%), rolls-in, and redirect rates, and differ in lateral stretch (4.52% against 2.05%), which accounts for the visible part of their 2.1-point gap. `gecko analyze --words` goes further and ranks real words by their share of a metric's cost. For Graphite on the `shai` word corpus, the lateral-stretch cost is dominated by one word: "you" carries 21% of it and "your" another 12%. A designer who sees this can decide whether the metric or the layout is at fault, which a single score cannot say.

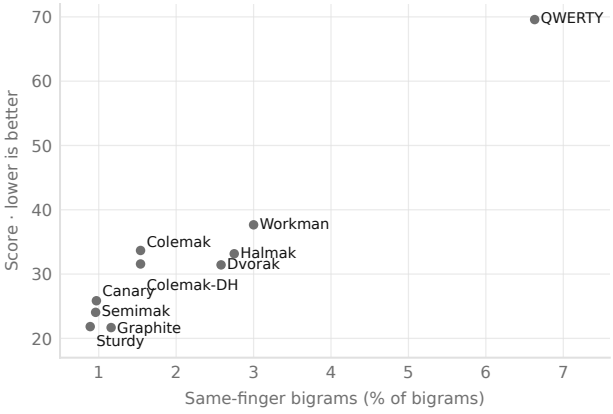


Figure 7: Reference layouts under the default profile (`gecko compare`, ANSI-30, English): score against same-finger bigram rate. Lower is better on both axes.

8.2 Design a layout for a board and a language

`gecko optimize` writes a run directory and prints the ranked layouts, the per-metric breakdown of the best, and a lower bound. On ANSI-30 with the default profile a 10 s search on 20 threads (52.6M evaluations, seed 7) found a layout scoring 17.561 against Graphite's 21.680 under the same profile. Figure 8 shows what it bought and what it paid: SFB falls from 1.16% to 0.51%

(the largest single gain), stretch, SFS, and movement improve, and the layout pays in redirects (4.7% against 2.7%) and effort. Seeing the price is the point of the breakdown: a different profile would pay differently.

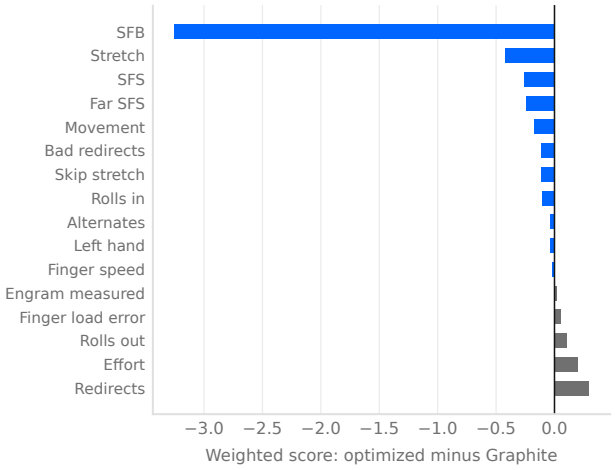


Figure 8: Per-metric weighted score of the optimized layout minus that of Graphite (same profile; metrics with non-zero weight). Left of zero is better. Run `20261022-031427-sa-8b3885fd`.

The same command designs for other boards. On the 36-key Corne with the five thumb keys pinned (`--pin '{space}{enter}…'`), 10 s of simulated annealing ran at 4.7M evaluations per second and had reached its final layout 2.7 s in, in this run; the bound there is as weak as on ANSI-30 (91% gap), so GECKO cannot certify that it is optimal. Scores on different boards are not comparable, because the geometry changes the cost tables. For programming, a layered Corne layout (symbol and number layers) is scored with the `code` profile and corpus, which add the layer-switch, symbol-bigram, and paired-bracket metrics.

A design is only as good as the corpus it was fitted to. `optimize --holdout shai` re-scores every ranked layout on a second corpus the search never sees. For the run above the best layout keeps 99.6% of its gain on the held-out corpus (start 70.2 → 18.3). The check also showed a limit: the top ten layouts span only 0.27 points on the search corpus (17.56 to 17.83), and their order has rank correlation 0.16 on the held-out one, so differences among near-ties should be settled by the breakdown, not by rank.

8.3 How good is “good”?

`gecko landscape` samples 2000 random layouts: scores are 98.4 ± 17.5, the random-walk correlation length is 8.5 swaps, and the best random layout scores 49.6. Of the six reference layouts it places among the samples, five lie more than 3.4 standard deviations below the random mean and no random sample beats them; QWERTY is 1.65 below, and 3.1% of random layouts beat it. The optimum we found (17.56) is 4.6 standard deviations below. `explore` then maps the good region: it archives local optima, groups them into families, and reports consensus slots. Because the archive keeps raw metric vectors, `atlas -p <profile>` re-ranks every optimum under another profile in milliseconds, so “would a different weighting change the winner?” is a query, not a new search.

8.4 See the price of a preference

Suppose a user wants more rolls. `gecko qd --axes roll_ratio,index_load` fills a grid with the best layout for each combination (Figure 9). In 20 s and 107M evaluations it filled 92 of 144 cells (64%). The best layout sits near 55% rolls and 27% index load. The best layout found with 65% rolls costs only 0.4 points more (17.93 against 17.56), but with 69% the cost is 2.6 (15%) and with 72% it is 5.5. The user chooses a point on the curve instead of accepting one optimum. `optimize --algo nsga2 --objectives B01,T03,M01` gives the same information as a Pareto front over named metrics.

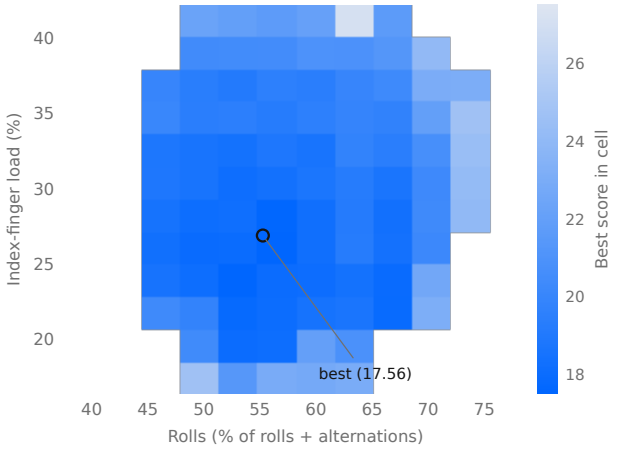


Figure 9: MAP-Elites map (ANSI-30, English, default profile, 20 s, 12×12 cells): best score per cell; blank cells were not reached. Run `20261022-031516-qd-af3fe56a`.

8.5 Try a layer or an adaptive key

Layers and adaptive (``magic'') keys are part of the layout model, so they are scored by the same tables. A magic key outputs a letter that depends on the previous one; `gecko magic` searches the rules greedily on a fixed layout, adding the rule that lowers the score most and dropping rules that stop paying. The bundled Magic Sturdy (one magic key, 14 rules) scores 19.995 against 21.831 for Sturdy, and SFB falls from 0.89% to 0.27%. The model charges one press for a magic output and has no typing data behind it, so this is a prediction to test, not a measurement. `optimize --from` a magic layout then moves the key like any symbol with the rules fixed, and the two commands can alternate.

8.6 Reproduce, exchange, and publish

Any run made with an evaluation budget can be re-run from its directory: `gecko verify run:<id>` checks the input hashes, re-executes with the recorded

parameters, and compares evaluations and every ranked layout. On a 3M-evaluation SA run it printed `reproduced`. Layouts move between tools: `layouts import` reads Oxeylyzer, genkey, and cmini files (reporting dropped keys as warnings), `layouts export` writes them back, `geometry import-kle` reads keyboard-layout-editor boards, and `layouts svg` draws the image the web Export tab produces, with finger colours or a frequency heat map. A GECKO score can therefore be checked against another tool's on the same layout, and a result can be shared as one file.

9. Related work

Table 4 summarizes the tools we read (details and sources in `research/studies/tool-comparison.md`; keynergy and `evolve-keyboard-layout` were read from documentation only and are omitted). They differ in what they judge and how they search. None of the layout analyzers we read scores a layout on an arbitrary 3D board with a hand model, none optimizes board and layout together, and only the integer-programming work of Karrenbauer and Oulasvirta certifies optimality; GECKO's Gilmore–Lawler bound is weaker (Section 6). GECKO's own claims are narrower: one inspectable objective shared by all searches, with per-metric explanations, maps of the landscape, and reproducible runs. For search we use ideas from QAP heuristics (robust tabu search [1], breakout local search [2]), Gilmore–Lawler bounds [3], multi-objective search [4], quality diversity [5], and CMA-ES [6]. For costs, we draw on finger independence [7], large-scale typing timing [8], and the Engram comfort study. Coverage of the academic literature for 2023–2026 is thin, so the final version must state what has not been measured by anyone.

Table 4: Tools compared (from documentation and source). "Geometry" is what the tool can score on.

Tool	Judges	Search	Geometry
carpalx	triad effort model	annealing	fixed 2D
Oxeylyzer	SFB/SFS, stretch, trigram classes	greedy, depth-N, annealing	fixed 2D
genkey	finger speed, LSB	greedy, multi-swap	fixed 2D
KLO	movement table, trigram irregularity	annealing, GA	fixed 2D
Kühlmak	bigram classes, balance	threshold annealing	fixed 2D
Engram	measured pair comfort	exhaustive, B&B	fixed 2D
Integer programming	ergonomics, familiarity	IP with bounds	fixed 2D
Cosmos, ergogen	none (generate boards)	manual, parametric	any
GECKO	48 metrics, hand model	8 optimizers, QD	2D and 3D

10. Reproducibility and engineering

Seeded RNG. xoshiro256++ is implemented in-tree and is identical on native and WebAssembly builds. **Run directories** (`runs/<time>-<algo>-<hash>/`) hold `config.json` (parameters, input names and content hashes, the full profile, start layout, pins, version) and `best.json` (top layouts with full breakdown, evaluations, speed, optimality gap). `gecko verify run:<id>` re-runs an evaluation-budget run and checks hashes, evaluation count, and best score; time-budget runs are refused because they are not deterministic. **Content hashing** (FNV-1a of geometry, corpus, profile tables, and layouts) de-duplicates results and detects stale atlases. **Stable JSON**: every CLI command supports `--json` with a versioned schema, and a contract test compares the web client's types against real server output. **CI** runs the Rust tests, clippy with warnings as errors, and rustdoc, plus the web type check, unit tests, the WASM parity test, and the contract test. Benchmarks append to `bench/history.jsonl` with the commit hash; curated experiments, including those in this paper, live in `bench/` with the scripts that produced them. **Size**: about 22.7k lines of Rust (19.4k in the core, including tests) and 7.6k of TypeScript. TODO(run): coverage.

11. Limitations and future work

Costs are models, not measurements: there is no user study yet, so scores are predicted comfort, not observed comfort or speed. The bound is weak on the full problem. A learned-profile experiment gained only 1.8 points of pairwise accuracy (53.8% against 52.0%), and community likes are confounded by popularity. Held-out corpus checks keep the gain but do not preserve the order of near-tied layouts (Section 8). Corpora are dominated by English; multilingual and code corpora exist but are less validated. The adaptive-key model is a prediction without typing data. All speed numbers come from one hybrid-core laptop CPU, and the thread-scaling points for 8 threads were taken with background load. Geometry search beyond flat boards (3D bilevel search, free-form keys, STL export) and learned timing costs are on the roadmap, not shipped.

12. Availability

GECKO is MIT-licensed (a LICENSE file is still to be added). Build with `cargo build --release`; run the web UI with `gecko serve`; the WASM package is built by `npm run wasm`. Corpora, boards, profiles, and reference layouts ship in `data/`. A first session:

```
gecko compare
gecko analyze graphite -c shai --words
gecko optimize --time 30s
gecko qd --axes roll_ratio,index_load
gecko explore --time 5m
gecko verify run:<id>
```

References

[1] É. Taillard. Robust taboo search for the quadratic assignment problem. *Parallel Computing*, 17, 1991.

[2] U. Benlic and J.-K. Hao. Breakout local search for the quadratic assignment problem. *Applied Mathematics and Computation*, 2013.

[3] P. C. Gilmore. Optimal and suboptimal algorithms for the quadratic assignment problem. *J. SIAM*, 10, 1962. E. L. Lawler. The quadratic assignment problem. *Management Science*, 9, 1963.

[4] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evolutionary Computation*, 6(2), 2002.

[5] J.-B. Mouret and J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015.

[6] N. Hansen and A. Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2), 2001.

[7] C. Häger-Ross and M. H. Schieber. Quantifying the independence of human finger movements. *J. Neuroscience*, 20(22), 2000.

[8] V. Dhakal, A. M. Feit, P. O. Kristensson, and A. Oulasvirta. Observations on typing from 136 million keystrokes. *Proc. CHI*, 2018.